

01 — Load Data

All the ways to load data you'll need in the exam.

Quick decision guide:

- CSV with header row → use `np.genfromtxt(..., skip_header=1)` or `pd.read_csv(...)`
- CSV / TXT without header → `pd.read_csv(..., header=None)`
- Need to split into X and t (target) → indexing examples below
- Need to normalize → Section 4
- Need to split train/val/test → Section 5

```
In [1]: import numpy as np
import pandas as pd
```

1. Load CSV with numpy — `np.genfromtxt`

Use when the exam says "you may use numpy".

- `skip_header=1` → skips the first row (column names)
- If no header, remove `skip_header`

```
In [8]: # --- CSV with header (e.g. train.csv has row: x0,x1,x2,x3,x4,t) ---
data = np.genfromtxt('train.csv', delimiter=',', skip_header=1)

print('Shape of full data:', data.shape)
print('First row:', data[0])
```

Shape of full data: (50, 6)

First row: [21.9892912 67.4374412 47.1311893 180.0403746 112.60746958
-69.68033551]

```
In [11]: # --- CSV without header ---
data = np.genfromtxt('train.csv', delimiter=',')
print('Shape of full data:', data.shape)
print('First row:', data[0])
print('Second row:', data[1])
```

Shape of full data: (51, 6)

First row: [nan nan nan nan nan nan]

Second row: [21.9892912 67.4374412 47.1311893 180.0403746 112.60746958
-69.68033551]

2. Load CSV with pandas — `pd.read_csv`

Easier to work with when the file has column names you want to use.

```
In [15]: # --- CSV with header ---
df = pd.read_csv('train.csv')
print(df.shape)
print(df.head(3))    #printing 1st 3 rows

# convert to numpy array for math
data = df.to_numpy()
print('As numpy:', data.shape)
```

```
(50, 6)
      x0      x1      x2      x3      x4      t
0  21.989291  67.437441  47.131189  180.040375  112.607470 -69.680336
1  18.424251  51.316419  38.671974  141.127230   86.560465 -11.893443
2   7.092821  19.718728  13.960253   53.116345   32.578957  24.949318
As numpy: (50, 6)
```

```
In [13]: # --- TXT or CSV without header ---
df = pd.read_csv('train.csv', header=None)
data = df.to_numpy()
print('As numpy:', data.shape)
```

```
As numpy: (51, 6)
```

3. Split data into X (features) and t / Y (target)

Most common exam patterns:

```
In [23]: # --- Pattern A: Last column is the target (regression) ---
# data shape is (m, n+1) where n = number of features

X = data[:, :-1]    # all rows, all columns except last → (m, n)
t = data[:, -1]     # all rows, last column           → (m,)

# OR explicitly by index (e.g. 5 features, 1 target):
X = data[:, :5]     # columns 0-4 → (m, 5)
t = data[:, 5]      # column 5    → (m,)

m, n = X.shape      # m = number of samples, n = number of features
print('X:', X.shape) # (50, 5)
print('t:', t.shape) # (50,)
```

```
X: (50, 5)
t: (50,)
```

```
In [19]: # --- Pattern B: Last column is binary label (classification) ---
# Y is usually reshaped to (m, 1) for matrix math in logistic regression

X = data[:, :5]     # features
```

```
Y = data[:, 2].reshape(m, 1) # Label as column vector → (m, 1)

print('X:', X.shape)    # (m, 2)
print('Y:', Y.shape)    # (m, 1)
```

X: (50, 5)

Y: (50, 1)

```
In [27]: # --- Pattern C: test file has no target column (just features) ---
X_test = np.genfromtxt('test.csv', delimiter=',', skip_header=1)
print('X_test:', X_test.shape)    # (10, 5)
```

X_test: (10, 5)

4. Normalize (Feature Scaling + Mean Normalization)

When to normalize: before PCA, before gradient descent (logistic/linear regression). **When NOT needed:** the exam data may already be in [-1, 1] (like Lab04).

Rule: ALWAYS compute mean and std from training data, then apply to test data.

```
In [24]: def featureNormalize(X):
    mean = np.mean(X, axis=0)    # mean of each column (each feature)
    std = np.std(X, axis=0)      # std of each column
    X_norm = (X - mean) / std
    return X_norm, mean, std

X_norm, mean, std = featureNormalize(X)

print('After normalization:')
print(' means:', X_norm.mean(axis=0).round(10), ' ← should be ~0')
print(' stds: ', X_norm.std(axis=0).round(10), ' ← should be ~1')
```

After normalization:

means: [0. 0. 0. -0. -0.] ← should be ~0

stds: [1. 1. 1. 1. 1.] ← should be ~1

```
In [28]: # Apply SAME mean/std to test data (never recompute from test!)
X_test_norm = (X_test - mean) / std
```

5. Add BIAS column (ones)

Required for logistic regression and linear regression with gradient descent. **NOT needed** when using the Normal Equation (it handles the intercept via the design matrix).

```
In [29]: # Add a column of ones at the front: X becomes [1, x1, x2, ...]
X_bias = np.hstack((np.ones((m, 1)), X_norm))    # (m, n+1)
```

```
print('X with bias:', X_bias.shape)
```

X with bias: (50, 6)

6. Reshape tricks

Common things you need fast:

```
In [30]: v = np.array([1, 2, 3, 4, 5])    # shape (5,) - 1D

# turn 1D into column vector (needed for matrix math)
col = v.reshape(5, 1)                    # shape (5, 1)
col = v.reshape(-1, 1)                   # same, -1 means 'figure it out'

# turn column vector back to 1D
flat = col.flatten()                     # shape (5,)

# turn 1D into row vector
row = v.reshape(1, 5)                    # shape (1, 5)

print('original:', v.shape)
print('column:   ', col.shape)
print('flat:     ', flat.shape)
print('row:      ', row.shape)
```

```
original: (5,)
column:   (5, 1)
flat:     (5,)
row:      (1, 5)
```

7. Manual train/validation split

If a question asks you to split the data yourself.

```
In [31]: # Suppose data has m samples, split 80% train / 20% validation (from both, train and target)
split = int(0.8 * m)                        # index where we cut

X_tr = X[:split]                            # first 80%
t_tr = t[:split]

X_val = X[split:]                          # last 20%
t_val = t[split:]

print('Train:', X_tr.shape, 'Val:', X_val.shape)
```

Train: (40, 5) Val: (10, 5)

Summary — which method to use?

Situation	Code
CSV with header, numpy-only	<code>np.genfromtxt(path, delimiter=',', skip_header=1)</code>
CSV/TXT no header, numpy-only	<code>np.genfromtxt(path, delimiter=',')</code>
CSV with header, pandas	<code>pd.read_csv(path).to_numpy()</code>
CSV no header, pandas	<code>pd.read_csv(path, header=None).to_numpy()</code>
Split features / target	<code>X = data[:, :n] , t = data[:, n]</code>
Binary label as column	<code>Y = data[:, -1].reshape(m, 1)</code>
Normalize	<code>(X - mean) / std</code> — always from training
Add bias	<code>np.hstack((np.ones((m,1)), X))</code>